Formal Algorithms for DeepSeek-R1

Formal Pseudocode in the Style of Phuong & Hutter (2022)

Based on: DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning" (arXiv:2501.12948, 2025)

February 27, 2026

Abstract

This document presents a formal pseudocode specification of the core algorithms in DeepSeek-R1, which incentivizes reasoning capability in large language models via reinforcement learning (RL). We formalize: (1) Group Relative Policy Optimization (GRPO), the RL algorithm that replaces PPO by estimating advantages from group-level reward statistics instead of a learned value model; (2) the rule-based and model-based reward functions for accuracy, format, language consistency, helpfulness, and safety; (3) the multi-stage training pipeline comprising cold-start SFT, reasoning-focused RL, rejection sampling with SFT on mixed data, and preference-aligned RL; and (4) the structured inference procedure with chain-of-thought reasoning. All notation is defined in Appendix A.

Contents

1 Introduction **2**

2 Notation and Conventions **2**

3 Core Components **2**

4 Reward Functions **3**

5 Training **4**

5.1 DeepSeek-R1-Zero: Pure RL from Base Model 4

5.2 DeepSeek-R1: Multi-Stage Pipeline 6

6 Inference **7**

A List of Notation **7**

1 Introduction

DeepSeek-R1 demonstrates that advanced reasoning capabilities—such as self-reflection, verification, and dynamic strategy adaptation—can emerge in large language models through pure reinforcement learning, without requiring human-annotated reasoning traces. The system is built upon DeepSeek-V3-Base (a 671B-parameter Mixture-of-Experts model) and trained using Group Relative Policy Optimization (GRPO).

The key insight is that by providing only outcome-based reward signals (correctness of final answers) and a minimal structural template (separating reasoning from answers via tags), the model autonomously develops sophisticated reasoning behaviors including increasing chain-of-thought length, self-correction ("aha moments"), and exploration of alternative solution strategies.

This document formalizes seven core algorithms:

- **Algorithm 1**: GRPO advantage estimation from grouped outputs.
- **Algorithm 2**: The GRPO policy optimization objective.
- **Algorithm 3**: Rule-based reward computation (accuracy + format + language consistency).
- **Algorithm 4**: Model-based reward computation (helpfulness + safety).
- **Algorithm 5**: DeepSeek-R1-Zero training (pure RL from base model).
- **Algorithm 6**: DeepSeek-R1 multi-stage training pipeline.
- **Algorithm 7**: Structured inference with chain-of-thought generation.

2 Notation and Conventions

We use the following conventions throughout this document. Let V denote a finite vocabulary with $N_V = |V|$ elements. Sequences of tokens are written $\boldsymbol{x} \equiv x[1:\ell] \equiv x[1]x[2]\ldots x[\ell] \in V^*$. Arrays are 1-indexed: $x[1:\ell]$ includes $x[\ell]$. $\boldsymbol{\theta}$ denotes all learnable parameters of the policy model; $\hat{\boldsymbol{\theta}}$ denotes trained parameters. $\pi_{\boldsymbol{\theta}}(o \mid q)$ denotes the probability of generating output sequence o given question q under policy parameterized by $\boldsymbol{\theta}$. We write $\boldsymbol{\theta}_{\text{old}}$ for the parameters of the sampling (old) policy, $\boldsymbol{\theta}_{\text{ref}}$ for the reference policy, and $\boldsymbol{\theta}$ for the policy being optimized.

For the reward system, $r_i \in \mathbb{R}$ denotes the scalar reward assigned to the i-th output in a group. The advantage A_i is computed by normalizing rewards within a group. The ratio $\rho_i = \pi_{\boldsymbol{\theta}}(o_i \mid q)/\pi_{\boldsymbol{\theta}_{\text{old}}}(o_i \mid q)$ is the importance sampling ratio.

3 Core Components

This section formalizes the two building blocks unique to DeepSeek-R1's training: the GRPO advantage estimation and the GRPO policy update objective.

Algorithm 1: $\{A_i\}_{i=1}^{G} \leftarrow$ **GRPOAdvantage**$(\{r_i\}_{i=1}^{G})$

/ Compute group-relative advantages by normalizing rewards within a group. */*
Input: $\{r_i\}_{i=1}^{G} \in \mathbb{R}^G$, rewards for G sampled outputs
Output: $\{A_i\}_{i=1}^{G} \in \mathbb{R}^G$, normalized advantage estimates

$\mu \leftarrow \frac{1}{G}\sum_{i=1}^{G} r_i$ */* group mean */*
$\sigma \leftarrow \sqrt{\frac{1}{G}\sum_{i=1}^{G}(r_i-\mu)^2}$ */* group std */*
for $i = 1, 2, \ldots, G$ **do**
  $A_i \leftarrow \dfrac{r_i - \mu}{\sigma}$
end
return $\{A_i\}_{i=1}^{G}$

Unlike PPO, which requires a learned value model of similar size to the policy to estimate advantages via Generalized Advantage Estimation (GAE), GRPO estimates advantages purely from the group statistics of sampled rewards. This eliminates the memory and computational overhead of training a separate value model—a critical advantage when training 671B-parameter models.

Algorithm 2: $\mathcal{J} \leftarrow$ **GRPOObjective**$(q, \{o_i\}_{i=1}^{G}, \{A_i\}_{i=1}^{G} \mid \boldsymbol{\theta}, \boldsymbol{\theta}_{\text{old}}, \boldsymbol{\theta}_{\text{ref}})$

/ Compute the GRPO clipped surrogate objective with KL regularization. */*
Input: $q \in V^*$, input question
Input: $\{o_i\}_{i=1}^{G} \in (V^*)^G$, sampled outputs from $\pi_{\boldsymbol{\theta}_{\text{old}}}$
Input: $\{A_i\}_{i=1}^{G} \in \mathbb{R}^G$, group-relative advantages (Algorithm 1)
Output: $\mathcal{J} \in \mathbb{R}$, scalar objective to maximize

Parameters: $\boldsymbol{\theta}$ (policy being optimized)
Frozen: $\boldsymbol{\theta}_{\text{old}}$ (sampling policy), $\boldsymbol{\theta}_{\text{ref}}$ (reference policy)
Hyperparameters: $\varepsilon \in (0, \infty)$ clip ratio, $\beta \in [0, \infty)$ KL coefficient

$\mathcal{J} \leftarrow 0$
for $i = 1, 2, \ldots, G$ **do**
  $\rho_i \leftarrow \dfrac{\pi_{\boldsymbol{\theta}}(o_i \mid q)}{\pi_{\boldsymbol{\theta}_{\text{old}}}(o_i \mid q)}$ */* importance sampling ratio */*
  $\mathcal{J}_i^{\text{clip}} \leftarrow \min\Big(\rho_i\, A_i,\ \text{clip}(\rho_i,\ 1-\varepsilon,\ 1+\varepsilon)\, A_i\Big)$
  / Unbiased KL divergence estimator (Schulman, 2020) */*
  $\hat{D}_i \leftarrow \dfrac{\pi_{\boldsymbol{\theta}_{\text{ref}}}(o_i \mid q)}{\pi_{\boldsymbol{\theta}}(o_i \mid q)} - \log \dfrac{\pi_{\boldsymbol{\theta}_{\text{ref}}}(o_i \mid q)}{\pi_{\boldsymbol{\theta}}(o_i \mid q)} - 1$
  $\mathcal{J} \leftarrow \mathcal{J} + \mathcal{J}_i^{\text{clip}} - \beta\, \hat{D}_i$
end
return $\mathcal{J}/G$

Key differences from PPO: (i) advantages are estimated from group reward statistics rather than a learned value model; (ii) KL divergence is added directly to the loss rather than as a per-token reward penalty, which avoids implicitly penalizing response length; (iii) the reference policy $\boldsymbol{\theta}_{\text{ref}}$ is periodically updated to the latest policy every N_{ref} steps.

4 Reward Functions

DeepSeek-R1 uses two classes of rewards: rule-based rewards for verifiable reasoning tasks, and model-based rewards for general helpfulness and safety.

Algorithm 3: $r \leftarrow \textbf{RuleReward}(q, o, a^* \mid \textbf{type})$

/ Compute rule-based reward for reasoning tasks. */*
Input: $q \in V^*$, question prompt
Input: $o \in V^*$, model output (containing $\langle$think$\rangle$ and $\langle$answer$\rangle$ tags)
Input: $a^* \in V^*$, ground-truth answer
Input: type $\in \{\text{math}, \text{code}, \text{stem}, \text{logic}\}$, task type
Output: $r \in \mathbb{R}$, scalar reward

/ Accuracy reward */*
$\hat{a} \leftarrow \textsc{ExtractAnswer}(o)$ */* extract from $\langle$answer$\rangle$ tags */*
if type = math **then**
  $r_{\text{acc}} \leftarrow \mathbb{I}[\textsc{SymPyMatch}(\hat{a}, a^*)]$ */* symbolic comparison */*
else if type = code **then**
  $r_{\text{acc}} \leftarrow \mathbb{I}[\textsc{PassAllTests}(\hat{a}, \mathcal{T}_q)]$ */* compiler + test cases $\mathcal{T}_q$ */*
else
  $r_{\text{acc}} \leftarrow \mathbb{I}[\hat{a} = a^*]$ */* exact match for STEM/logic */*
end

/ Format reward */*
$r_{\text{fmt}} \leftarrow \mathbb{I}[o$ contains valid $\langle$think$\rangle$...$\langle$/think$\rangle$ and $\langle$answer$\rangle$...$\langle$/answer$\rangle]$

/ Language consistency reward */*
$r_{\text{lang}} \leftarrow \dfrac{|\{w \in o : \textsc{Lang}(w) = \textsc{Lang}(q)\}|}{|\{w \in o\}|}$ */* proportion of target-language words */*
return $r = r_{\text{acc}} + r_{\text{fmt}} + r_{\text{lang}}$

Notably, neural reward models (whether outcome-based or process-based) are deliberately *not* used for reasoning tasks, because they are susceptible to reward hacking during large-scale RL. Rule-based rewards provide reliable, non-exploitable training signals.

Algorithm 4: $r \leftarrow \textbf{ModelReward}(q, o \mid \hat{\boldsymbol{\theta}}_{\text{help}}, \hat{\boldsymbol{\theta}}_{\text{safe}}, \textbf{dataset})$

/ Compute model-based reward for general (non-reasoning) tasks. */*
Input: $q \in V^*$, user query
Input: $o \in V^*$, model response
Output: $r \in \mathbb{R}$, scalar reward
Frozen models: $\hat{\boldsymbol{\theta}}_{\text{help}}$ (helpful RM), $\hat{\boldsymbol{\theta}}_{\text{safe}}$ (safety RM)

if dataset = helpfulness **then**
  $o_{\text{summary}} \leftarrow \textsc{ExtractSummary}(o)$ */* assess only final summary */*
  $r_{\text{pref}} \leftarrow \text{RM}_{\text{help}}(o_{\text{summary}} \mid \hat{\boldsymbol{\theta}}_{\text{help}})$ */* scalar preference score */*
else if dataset = safety **then**
  $r_{\text{pref}} \leftarrow \text{RM}_{\text{safe}}(o \mid \hat{\boldsymbol{\theta}}_{\text{safe}})$ */* point-wise safe/unsafe score on full response */*
end

$r_{\text{fmt}} \leftarrow \mathbb{I}[o$ has valid format$]$
return $r = r_{\text{pref}} + r_{\text{fmt}}$

5 Training

5.1 DeepSeek-R1-Zero: Pure RL from Base Model

DeepSeek-R1-Zero applies GRPO directly to the base model without any supervised fine-tuning, demonstrating that reasoning can emerge from RL alone.

Algorithm 5: $\hat{\boldsymbol{\theta}} \leftarrow$ **TrainR1Zero**$(\mathcal{Q}, \boldsymbol{\theta}_0)$

```
/* Train DeepSeek-R1-Zero via pure RL on the base model.                    */
Input: Q, set of reasoning questions with ground-truth answers
Input: θ_0, pre-trained base model parameters (DeepSeek-V3-Base)
Output: θ̂, trained policy parameters
Hyperparameters: N_steps = 10,400, G = 16 (group size), B = 32 (questions/step),
  η = 3 × 10^-6 (learning rate), β = 0.001 (KL coefficient),
  ε (clip ratio; not specified for R1-Zero), τ_sample = 1.0 (rollout temperature),
  ℓ_max ∈ {32768, 65536} (max output length), N_ref = 400 (ref. update interval)
θ ← θ_0
θ_ref ← θ_0
for s = 1, 2, ..., N_steps do
    Sample mini-batch {q_b}_{b=1}^B ~ Q
    if s > 8200 then
        ℓ_max ← 65536
    end
                                                   /* extend max length at step 8.2k */
    /* Rollout phase: sample G outputs per question                             */
    θ_old ← θ
    for b = 1, ..., B do
        for i = 1, ..., G do
            o_i^(b) ~ π_θ_old(· | Template(q_b)) with temperature τ_sample, max length ℓ_max
        end
    end
    /* Reward computation                                                       */
    for b = 1, ..., B do
        for i = 1, ..., G do
            r_i^(b) ← RuleReward(q_b, o_i^(b), a*_b | type_b)     /* Algorithm 3, without r_lang */
        end
        {A_i^(b)}_{i=1}^G ← GRPOAdvantage({r_i^(b)}_{i=1}^G)                  /* Algorithm 1 */
    end
    /* Policy update (single inner epoch, 16 mini-batches)                      */
    J ← (1/B) Σ_{b=1}^B GRPOObjective(q_b, {o_i^(b)}, {A_i^(b)} | θ, θ_old, θ_ref)
    θ ← θ + η · ∇_θ J
    if s mod N_ref = 0 then
        θ_ref ← θ                                   /* periodically refresh reference policy */
    end
end
return θ̂ = θ
```

The Template function wraps each question q in the prompt from Table 1 of the paper: "*A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within* `<think>`...`</think>` *and* `<answer>`...`</answer>` *tags, respectively, i.e.,* `<think>` *reasoning process here* `</think>` `<answer>` *answer here* `</answer>`*. User: {prompt}. Assistant:*" This avoids content-specific biases on the reasoning process. Note that the clip ratio ε is not explicitly stated for R1-Zero in the paper; it is specified as $\varepsilon = 10$ only for the R1 first-stage RL (Section 3.2.1).

5.2 DeepSeek-R1: Multi-Stage Pipeline

DeepSeek-R1 extends R1-Zero through a four-stage pipeline to address readability, language mixing, and general-purpose capabilities.

Algorithm 6: $\hat{\boldsymbol{\theta}}_{\text{R1}} \leftarrow \textbf{TrainR1}(\mathcal{Q}_{\text{reason}}, \mathcal{Q}_{\text{general}}, \mathcal{D}_{\text{cold}}, \mathcal{D}_{\text{SFT}}, \boldsymbol{\theta}_0)$

```
/* Full DeepSeek-R1 multi-stage training pipeline. */
Input: Q_reason, reasoning prompts (math, code, STEM, logic)
Input: Q_general, general prompts (helpfulness + safety)
Input: D_cold, cold-start long-CoT data (~thousands of examples)
Input: D_SFT, 800k SFT dataset (600k reasoning + 200k non-reasoning)
Input: θ_0, DeepSeek-V3-Base parameters
Output: θ̂_R1, final trained parameters

/* Stage 1: Cold-Start SFT */
θ_Dev1 ← SFT(θ_0, D_cold)                    /* 2–3 epochs, η_0 = 5×10^-5 */
/* Stage 2: Reasoning-Focused RL (first RL stage) */
θ_Dev2 ← GRPO-Train(θ_Dev1, Q_reason)
  with rewards: r = RuleReward(q, o, a*) + r_lang
  and hyperparameters: ε = 10, τ = 1.0, ℓ_max = 32768
/* Stage 3: Rejection Sampling + Mixed SFT */
/* Generate reasoning trajectories from θ_Dev2 via rejection sampling */
for each q ∈ Q_reason do
    Sample {o_j}_{j=1}^M ~ π_θDev2(· | q) with τ = 1.0
    Retain o_j if correct answer ∧ readable format ∧ consistent language
    Refine retained outputs via DeepSeek-V3 for formatting and readability
end
D_SFT ← filtered reasoning data ∪ non-reasoning SFT data
θ_Dev3 ← SFT(θ_0, D_SFT)                     /* train from base on 800k mixed data */
/* Stage 4: Comprehensive RL (second RL stage) */
θ̂_R1 ← GRPO-Train(θ_Dev3, Q_reason ∪ Q_general)
  with rewards: r = r_reasoning + r_general + r_lang
  where r_reasoning = RuleReward(·) (Algorithm 3)

r_general = ModelReward(·) (Algorithm 4)
  and hyperparameters: τ = 0.7, 1700 total steps, general data in last 400 steps
return θ̂_R1
```

The pipeline produces intermediate checkpoints Dev1 through Dev3. Dev1 has improved readability but slightly degraded reasoning (due to the small cold-start dataset). Dev2 recovers and improves reasoning via RL. Dev3 incorporates broad capabilities through mixed SFT. The final R1 model refines all capabilities through comprehensive RL, with general instruction-following data introduced only in the final 400 steps to limit reward hacking.

6 Inference

Algorithm 7: $(\boldsymbol{c}, \boldsymbol{a}) \leftarrow$ **R1Inference**$(\boldsymbol{x} \mid \hat{\boldsymbol{\theta}})$

/* *Generate a structured response with chain-of-thought reasoning followed by an answer.* */

Input: $\boldsymbol{x} \in V^*$, user prompt
Output: $\boldsymbol{c} \in V^*$, chain-of-thought reasoning (within ⟨think⟩ tags)
Output: $\boldsymbol{a} \in V^*$, final answer (within ⟨answer⟩ tags)
Trained parameters: $\hat{\boldsymbol{\theta}}$
Hyperparameters: $\tau \in (0, \infty)$ temperature, $p_{\text{top}} \in (0, 1]$ nucleus threshold,
  $\ell_{\max} = 32768$ max generation tokens

$\boldsymbol{z} \leftarrow$ Template$(\boldsymbol{x})$ /* *wrap in conversation template* */
$\boldsymbol{y} \leftarrow$ `<think>` /* *begin chain-of-thought* */
for $t = 1, 2, \ldots, \ell_{\max}$ **do**
  $\boldsymbol{p} \leftarrow \pi_{\hat{\boldsymbol{\theta}}}(\cdot \mid [\boldsymbol{z}, \boldsymbol{y}])$ /* *next-token distribution* */
  Sample y_t from $\boldsymbol{p}^{1/\tau}$ with nucleus sampling (top-p_{top})
  $\boldsymbol{y} \leftarrow [\boldsymbol{y}, y_t]$
  if $y_t =$ `EOS` **or** $\boldsymbol{y}$ ends with `</answer>` **then**
    break
  end
end
$\boldsymbol{c} \leftarrow$ tokens between `<think>` and `</think>` in $\boldsymbol{y}$
$\boldsymbol{a} \leftarrow$ tokens between `<answer>` and `</answer>` in $\boldsymbol{y}$
return $(\boldsymbol{c}, \boldsymbol{a})$

DeepSeek-R1 dynamically allocates test-time compute based on problem difficulty: simple questions may use fewer than 100 thinking tokens, while competition-level mathematics problems can consume over 18,000 thinking tokens. The model is sensitive to prompting—zero-shot prompting is recommended, as few-shot prompting consistently degrades performance.

A List of Notation

Symbol	Type	Explanation
		Index sets and sizes
$[N]$	$:= \{1, \ldots, N\}$	set of integers 1 to N
$V \cong [N_V]$		vocabulary
$N_V \in \mathbb{N}$		vocabulary size
$G \in \mathbb{N}$		group size (outputs per question)
$B \in \mathbb{N}$		questions per training step (batch)
		Sequences
$q \in V^*$		input question / prompt
$o, o_i \in V^*$		generated output sequence
$\boldsymbol{x} \equiv x[1:\ell]$	$\in V^\ell$	token sequence
$a^* \in V^*$		ground-truth answer
$\hat{a} \in V^*$		extracted predicted answer
$\boldsymbol{c} \in V^*$		chain-of-thought reasoning tokens
$\boldsymbol{a} \in V^*$		final answer tokens

Symbol	Type	Explanation
		Datasets and collections
$\mathcal{Q}$		set of questions with ground-truth answers
$\mathcal{Q}_{\text{reason}}$		reasoning prompts (math, code, STEM, logic)
$\mathcal{Q}_{\text{general}}$		general prompts (helpfulness + safety)
$\mathcal{D}_{\text{cold}}$		cold-start long-CoT SFT data
$\mathcal{D}_{\text{SFT}}$		mixed SFT dataset ($\sim$800k samples)
$\mathcal{T}_q$		set of test cases for code problem q
$M \in \mathbb{N}$		number of samples for rejection sampling
		Policy and probability
$\pi_{\boldsymbol{\theta}}(o \mid q)$	$\in [0,1]$	probability of output o given q under $\boldsymbol{\theta}$
$\boldsymbol{\theta} \in \mathbb{R}^d$		learnable parameters of the policy model
$\hat{\boldsymbol{\theta}} \in \mathbb{R}^d$		trained (final) parameters
$\boldsymbol{\theta}_0 \in \mathbb{R}^d$		pre-trained base model parameters
$\boldsymbol{\theta}_{\text{old}} \in \mathbb{R}^d$		old (sampling) policy parameters
$\boldsymbol{\theta}_{\text{ref}} \in \mathbb{R}^d$		reference policy parameters
$\rho_i \in (0,\infty)$		importance sampling ratio $\pi_{\boldsymbol{\theta}}(o_i\|q)/\pi_{\boldsymbol{\theta}_{\text{old}}}(o_i\|q)$
		Rewards and advantages
$r_i \in \mathbb{R}$		scalar reward for i-th output
$r_{\text{acc}} \in \{0,1\}$		accuracy reward (correct answer)
$r_{\text{fmt}} \in \{0,1\}$		format reward (valid tags)
$r_{\text{lang}} \in [0,1]$		language consistency reward
$r_{\text{pref}} \in \mathbb{R}$		preference reward from reward model
$A_i \in \mathbb{R}$		group-relative advantage for i-th output
		Hyperparameters
$\eta \in (0,\infty)$		learning rate
$\varepsilon \in (0,\infty)$		GRPO clipping ratio
$\beta \in [0,\infty)$		KL divergence penalty coefficient
$\tau_{\text{sample}} \in (0,\infty)$		rollout sampling temperature
$\tau \in (0,\infty)$		inference temperature
$\ell_{\max} \in \mathbb{N}$		maximum output token length
$N_{\text{steps}} \in \mathbb{N}$		total training steps
$N_{\text{ref}} \in \mathbb{N}$		reference model update interval (steps)
$p_{\text{top}} \in (0,1]$		nucleus sampling threshold
		Operators and functions
$\text{clip}(x,a,b)$	$\in [a,b]$	clamp x to $[a,b]$
$\hat{D}_i$	$\in \mathbb{R}$	unbiased KL divergence estimator for output i
$\mathbb{I}[\cdot]$	$\in \{0,1\}$	indicator function
$\textsc{Lang}(w)$		detected language of word w
RM_{help}		helpful reward model
RM_{safe}		safety reward model

References

- DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.* arXiv:2501.12948v2, January 2025.

- DeepSeek-AI. *DeepSeek-V3 Technical Report.* arXiv:2412.19437, December 2024.
- Shao, Z. et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.* arXiv:2402.03300, 2024. (Introduces GRPO.)
- Schulman, J. et al. *Proximal Policy Optimization Algorithms.* arXiv:1707.06347, 2017.
- Schulman, J. *Approximating KL Divergence.* Blog post, 2020.
- Phuong, M. & Hutter, M. *Formal Algorithms for Transformers.* arXiv:2207.09238, 2022.